

# Laboratório 4

## Ponteiros e Funções (cont.)

ICP141 - Programação de Computadores II

Prof. Marcos Tomazzoli Leipnitz  
marcosleipnitz@ic.ufrj.br

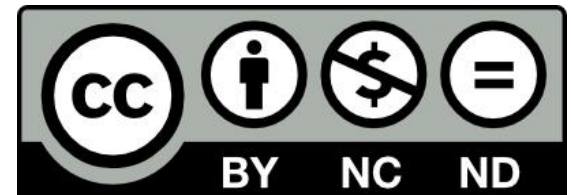


**UFRJ**

UNIVERSIDADE FEDERAL  
DO RIO DE JANEIRO



INSTITUTO DE  
COMPUTAÇÃO  
UFRJ



# Exercício 1

O que o programa ao lado faz, caso você digite o seu primeiro e último nomes separados por um espaço em branco? Ele pode acessar regiões de memória indevidamente? Em quais situações? **Corrija-o** para que isto não aconteça.

```
#include <stdio.h>
void misterio (char *n);
int main (void)
{
    char nome [41];
    gets(nome);
    misterio(nome);
    return 0;
}

void misterio (char *n)
{
    while (*n != ' ')
        n++;
    puts(n);
}
```

# Exercício 2 - Exemplo Resolvido

Construir uma função **inicM()** que inicializa uma matriz quadrada com 9 elementos e outra função **mostraM()** que mostra os elementos dessa matriz.

# Exercício 2 - Exemplo Resolvido

```
#include <stdio.h>
#define DIM 3
void inicM(int s[ ][DIM])
{
    int i, j;
    for (i=0;i<DIM;i++)
        for (j=0; j <DIM;j++)
            s[i][j] = 0;
}
void mostraM (int s[DIM][DIM])
{
    int i, j;
    for (i=0; i<DIM;i++)
    {
        for (j=0;j<DIM;j++)
            printf(" [%d] ", s[i][j]);
        printf("\n");
    }
}
int main()
{
    int s[DIM][DIM];
    inicM(s);
    mostraM (s);
    return 0;}
```

Vetores para ponteiros:

`* (s[i]+j)`

Ponteiros para ponteiros:

`* (* (s+i)+j)`

Saída:

|     |     |     |
|-----|-----|-----|
| [0] | [0] | [0] |
| [0] | [0] | [0] |
| [0] | [0] | [0] |

# Exercício 2

Adicione uma nova função **incluiElem(...)** ao exemplo anterior, cujos parâmetros são **s**, **i**, **j** e **num**, tal que esta modifique o elemento **s[i][j]** com o valor **num**. Faça um programa principal que teste a função **incluiElem(...)** para **i=1**, **j=2** e **num=10**.

# Exercício 3

Considere que o programa abaixo foi compilado (**exec3.out**) e será executado via linha de comando da seguinte forma:

**./exec3.out 35**

Qual será a saída apresentada?

```
#include <stdio.h>
#include <stdlib.h>
int k=0;
int main(int argc, char
*argv[])
{
    if(argc==1)
        return -1;
    k=atoi(argv[1]);
    int vetor[]={
10,20,30,40,50,60};
    int *p1, *p2, *p3;
    int i = 100, j;
    p1 = &vetor[2];
    p2 = &i;
    printf("%d, %d\n", *p1, *p2);
    p3=&k;
```

```
for (j=3;j>0;j--)
{
    int k;
    if (j%2!=0)
    {
        k=j*5;
        printf("%d, %d\n", j, k);
    }
    *p3=k;
    *p2=*p3;
    p1++;
}
printf("%d, %d, %d\n", *p1, *p2, *p3);
printf("%d, %d, %d\n", i, j, k);
return 0;
}
```

# Desafio 3

Implemente uma função chamada **posicao**, com o seguinte protótipo:

```
int posicao(char *substr, char *str);
```

Essa função deve verificar se a cadeia de caracteres apontada por **substr** está contida na cadeia apontada por **str**. Caso a substring ocorra dentro da string, a função deve retornar a posição (baseada em índice, começando de 0) onde se inicia a primeira ocorrência de **substr** em **str**. Se a substring não for encontrada, a função deve retornar **-1**.

## Requisitos:

1. A função deve retornar apenas a **posição da primeira ocorrência**, mesmo que a substring apareça mais de uma vez na string.
2. Faça também um programa principal (**main**) que permita testar a função, lendo as duas strings do usuário e imprimindo o resultado da busca.
3. Não utilize a biblioteca **<string.h>** ou qualquer função pronta de busca. A função deve ser implementada manualmente, usando apenas notação de ponteiros (sem colchetes [ ]).

# Desafio 3

## Exemplos de entradas e saídas:

String: "programacao em C"  
Substring: "ama"  
Substring encontrada na posição 5.

String: "banana"  
Substring: "na"  
Substring encontrada na posição 2.

String: "computador"  
Substring: "xyz"  
Substring não encontrada.